JOY UNIVERSITY

Established vide Tamil Nadu State Pvt. Universities Act 2019

SUBJECT: COMPUTER ORGANISATION

ASSIGNMENT -1

TOPIC: THE INSTRUCTION SETS IN COMPUTER ORGANIZATION

SUBMITTED BY :

Name: Ch. Prakash Chowdary

PRN:2024BTCY043

SUBMITTED TO :

Mrs. Pavitha

SOCI

The Architectural Foundations of Instruction Sets in Computer Organization

The instruction set architecture (ISA) serves as the primary conceptual bridge between a computer's hardware components and the software programs that utilize them for functional execution. In the hierarchy of computer organization, the ISA represents an abstract model of a computer, often described as the "programmer's view" of the machine. It defines the specific operations a processor is capable of performing, the data types it can manipulate, the register sets available for temporary storage, and the mechanisms by which it interacts with main memory and peripheral devices. This architectural blueprint functions as a rigid contract between the hardware designer and the software developer; as long as the hardware correctly implements the instructions defined in the ISA, any software compiled for that ISA will execute correctly, regardless of the underlying micro-architectural complexities.

The Functional Role of the Instruction Set Architecture

The significance of the ISA lies in its role as a standardized interface. It provides the necessary abstraction that allows software to be written and compiled independently of the specific physical implementation of the processor. For example, the same machine code can run on a variety of processors within a family, such as different generations of the x86 or ARM architectures, even if those processors differ significantly in clock speed, power consumption, or internal design techniques like pipelining and superscalar execution. This binary compatibility is essential for the economic viability of software ecosystems, as it prevents the need for recompiling or rewriting applications every time a new processor is released.

Beyond simple command execution, the ISA defines the organizational state of the computer. This includes the number and size of general-purpose registers, the structure of the program counter (PC), and the status registers that track conditions like carry, overflow, or zero results. It also specifies the address space—the total number of unique memory locations the processor can access—and the addressability of that memory, which refers to the number of bits stored at each address, typically eight bits or one byte in modern systems. The ISA further dictates how the operating system interacts with the hardware to manage resources through virtual memory, protection schemes, and interrupt handling.

Distinguishing ISA from Microarchitecture

A fundamental distinction in computer organization is the separation of the instruction set architecture from the microarchitecture. While the ISA defines *what* a processor does, the microarchitecture defines *how* those operations are physically realized. Multiple microarchitectures can implement the same ISA; for instance, a highperformance server processor and a low-power mobile processor might both adhere to the ARM v8.6 architecture, but their internal designs—such as the number of execution units, the size of cache memory, and the complexity of branch prediction logic—will be drastically different.

This separation is often illustrated using the analogy of an automobile's interface. The "ISA" of a car consists of the steering wheel, the brake and accelerator pedals, and the gear shift—the components the driver must understand to operate the vehicle. The "microarchitecture" is the engine under the hood, whether it is a four-cylinder, a V8, or an electric motor. The driver (software) does not need to know the engine's internal workings as long as pressing the accelerator (executing an instruction) results in the expected forward motion. This abstraction allows hardware engineers to innovate at the circuit level, improving speed and efficiency, without breaking the existing software base.

Fundamental Components of a Machine Instruction

Every machine instruction is an atomic unit of processing, meaning it is either executed in its entirety or not at all, ensuring the processor remains in a well-defined state. Instructions are represented as a sequence of bits, which are divided into distinct groups called fields. These fields provide the control unit with the information necessary to fetch, decode, and execute the command.

Operation Code (Opcode)

The operation code, or opcode, is the mandatory portion of an instruction that identifies the specific task to be performed. Opcodes represent actions such as addition, subtraction, data movement, or logical comparisons. The bit pattern of the opcode is interpreted by the control unit's decoder, which then triggers the appropriate sequence of micro-operations within the processor's internal logic.

The length of the opcode field establishes the maximum number of unique instructions a processor can support. If an ISA allocates 4 bits for the opcode, it can define $2^4 = 16$ distinct operations. Modern processors typically use much larger or variable-length opcodes to support hundreds of specialized instructions. In architectures like the LC-3b, opcodes are fixed at bits, allowing for a uniform decoding process that simplifies the hardware design. In contrast, the x86 architecture uses variable-length opcodes that can be modified by prefixes, reflecting its evolution over decades of architectural expansion.

Operands and Operand Fields

Operands are the data elements or memory locations upon which the opcode acts. An instruction may contain zero, one, two, or three operand fields, depending on the architecture and the specific operation.

- **Registers:** Many instructions operate on data stored in internal general-purpose registers. The number of registers is limited by the size of the register identification field; for example, a 3-bit field can address $2^3 = 8$ registers.
- **Immediate Values:** Some instructions include the actual data to be used—such as a constant number—directly within the instruction bits. This is known as an immediate operand.
- **Memory Addresses:** Instructions that transfer data to or from RAM must include a field to specify the memory address or a method to calculate that address.

Addressing Mode Field

The addressing mode field provides the rules for interpreting the operand fields. It tells the CPU whether the operand field contains the actual data, a direct memory address, a register number, or a pointer to another location. By using different addressing modes, a single opcode like "ADD" can be used to add a constant to a register, add the contents of two registers, or add a value from memory to a register.

Field Type	Purpose	Impact on Computer Organization
Opcode	Specifies the operation (ADD, MOV, etc.)	Determines the complexity of the control unit decoder.
Mode	Defines how to locate the operand	Enables advanced features like pointers and array indexing.
Operand	Provides data or an address	Influences instruction length and memory bandwidth requirements.

Classification of Instruction Set Architectures

The architectural complexity of an ISA is a primary factor in the design of a computer system. The two dominant paradigms are Complex Instruction Set Computing (CISC) and Reduced Instruction Set Computing (RISC), each representing a different philosophy regarding the division of labor between hardware and software.

Complex Instruction Set Computing (CISC)

The CISC design philosophy aims to minimize the number of instructions per program by providing a large and diverse set of instructions, some of which can

perform complex, multi-step operations. A single CISC instruction might handle memory access, arithmetic, and data storage in one command.

CISC Characteristics:

- **Variable Length:** Instructions can vary in size, which maximizes code density but makes instruction decoding more difficult for the hardware.
- **Complex Addressing Modes:** Supports many ways to access memory, reducing the need for separate load/store instructions.
- **Microprogrammed Control:** Because instructions are complex, they are often broken down internally into smaller, micro-coded steps.
- **Fewer Registers:** Since many operations can happen directly in memory, there is less pressure to have a large number of internal registers.

The x86 architecture is the most prominent example of CISC. It was developed when memory was expensive and programming was often done in assembly; having "powerful" instructions reduced the total memory footprint of the software.

Reduced Instruction Set Computing (RISC)

RISC architectures prioritize simplicity and speed by using a small set of simple, fixed-length instructions that execute in a single clock cycle. The philosophy is that a larger number of simple instructions, if executed very quickly, will outperform a smaller number of complex ones.

RISC Characteristics:

- **Fixed Length:** Instructions are usually 32 or 64 bits long, allowing the CPU to fetch and decode them much more efficiently.
- **Load/Store Architecture:** Only specific instructions can access memory. All arithmetic and logic must be performed on data already in registers.
- **Hardwired Control:** The simple instructions can be executed directly by hardware logic, avoiding the overhead of microcode.
- **Large Register File:** To compensate for the lack of memory-to-memory operations, RISC chips provide a large number of general-purpose registers.

Examples of RISC include ARM, MIPS, and RISC-V. These architectures are highly suitable for pipelining, where multiple instructions are processed in various stages simultaneously, similar to an assembly line.

Comparative Table: CISC vs. RISC

Feature	CISC	RISC
Instruction Set	Large, variable-length	Small, fixed-length
Addressing Modes	Complex and many	Simple and few

Memory Operations	Memory-to-memory	Load/Store only
Clock Cycles	Multiple cycles per instruction	Typically one cycle per instruction
Design Emphasis	Hardware complexity	Software/Compiler efficiency
Registers	Limited number of GPRs	Many general-purpose registers
Control Unit	Microprogrammed	Hardwired

Detailed Analysis of Instruction Formats

The number of address fields in an instruction format significantly impacts the complexity of the processor's internal organization and the length of the resulting programs. To compare these formats, we analyze how each evaluates the arithmetic expression $X = (A + B) \times (C + D)$.

Zero-Address Instruction Format

Zero-address instructions do not use any explicit address fields for operands. They are primarily found in stack-organized computer architectures, where operands are implicitly taken from the top of the stack (TOS). To evaluate an expression, it must be converted into Reverse Polish Notation (RPN), also known as Postfix notation.

Mechanism and Evaluation of $X = (A+B) \times (C+D)$:

In Postfix, the expression is $AB+CD+\times$.

1. PUSH A : The value at memory location A is placed on the stack.
2. PUSH B : The value at B is placed on the stack.
3. ADD : The top two values (A and B) are popped, added together, and the sum $(A+B)$ is pushed back onto the stack.
4. PUSH C : C is pushed onto the stack.
5. PUSH D : D is pushed onto the stack.
6. ADD : C and D are popped, added, and the sum $(C+D)$ is pushed back.
7. MUL : $(A+B)$ and $(C+D)$ are popped, multiplied, and the result is pushed.
8. POP X : The final result is removed from the stack and stored in memory location X.

Architectural Implications: This format is very compact because instructions like ADD do not need to specify any addresses. However, it requires a high number of total instructions, and the stack model can be a bottleneck for performance.

One-Address Instruction Format

One-address instructions specify a single operand, while the other operand is implicitly assumed to be in a special register called the Accumulator (AC). The result of the operation is also stored back into the Accumulator.

Mechanism and Evaluation of $X = (A+B) \times (C+D)$:

1. LOAD A : $AC \leftarrow M[A]$
2. ADD B : $AC \leftarrow AC + M$ (Accumulator now contains $A+B$)
3. STORE T : $M \leftarrow AC$ (Intermediate result $A+B$ is stored in temporary memory location T)
4. LOAD C : $AC \leftarrow M[C]$
5. ADD D : $AC \leftarrow AC + M$ (Accumulator now contains $C+D$)
6. MUL T : $AC \leftarrow AC \times M$ (Accumulator now contains $(A+B) \times (C+D)$)
7. STORE X : $M[X] \leftarrow AC$

Architectural Implications: One-address instructions are slightly larger than zero-address ones but result in fewer instructions for simple tasks. They were standard in early computers but are less flexible than register-based systems.

Two-Address Instruction Format

This is the most common format in commercial CISC computers like x86. It specifies two operands, usually one register and one memory location, or two registers. One of the address fields serves as both a source and the destination where the result is stored.

Mechanism and Evaluation of $X = (A+B) \times (C+D)$:

1. MOV R1, A : $R1 \leftarrow M[A]$ (Load A into register R1)
2. ADD R1, B : $R1 \leftarrow R1 + M$ (R1 now contains $A+B$)
3. MOV R2, C : $R2 \leftarrow M[C]$ (Load C into register R2)
4. ADD R2, D : $R2 \leftarrow R2 + M$ (R2 now contains $C+D$)
5. MUL R1, R2 : $R1 \leftarrow R1 \times R2$ (R1 now contains the final product)
6. MOV X, R1 : $M[X] \leftarrow R1$

Architectural Implications: Two-address instructions are more powerful and allow for more complex expressions to be handled with fewer total instructions than the accumulator model. However, since one operand is overwritten, additional MOV instructions are often required to preserve data.

Three-Address Instruction Format

Three-address instructions specify three distinct operands: two sources and one destination. This format is highly efficient for the compiler but requires very long instructions to encode all three addresses.

Mechanism and Evaluation of $X = (A+B) \times (C+D)$:

1. ADD R1, A, B : $R1 \leftarrow M[A] + M$
2. ADD R2, C, D : $R2 \leftarrow M[C] + M$
3. MUL X, R1, R2 : $M[X] \leftarrow R1 \times R2$

Architectural Implications: This format results in the shortest possible program in terms of instruction count. While it simplifies program logic, the sheer number of bits required per instruction increases memory bandwidth pressure.

Instruction Classification by Functionality

The instructions in a computer's ISA are categorized by the nature of the task they perform. A robust ISA must provide instructions in each of the following categories to be considered "general-purpose".

Data Transfer Instructions

These are the most frequently used instructions in any program, responsible for moving data between memory and registers or between registers.

- **Load/Store:** LOAD retrieves data from memory to a register, while STORE does the reverse.
- **Move:** MOV copies data from one register to another or from an immediate value into a register.
- **Stack Operations:** PUSH and POP manage data on the system stack, which is critical for function calls and local variables.

Arithmetic and Logical Instructions

These instructions perform the actual data manipulation using the Arithmetic Logic Unit (ALU).

- **Arithmetic:** Common operations include ADD, SUB, MUL, and DIV. Some ISAs also include specialized arithmetic like increment (INC) and decrement (DEC).
- **Logical:** Bitwise operations such as AND, OR, XOR, and NOT allow for masking and individual bit manipulation.
- **Shift and Rotate:** Instructions like SHL (Shift Left) and SHR (Shift Right) move bits within a register, effectively performing fast multiplication or division by powers of two.

Control Flow Instructions

Control instructions determine the sequence of program execution by modifying the Program Counter (PC). Without them, a computer would only be able to execute instructions in a single, linear pass.